Laboratorio 5 — Hoja de trabajo:
HTTPS con una CA propia y cabeceras de seguridad

Nombre: Leonardo Avila Bejarano **Fecha:** 12/10/2026 **Entorno:** [ ] VirtualBox [x] Docker

Asignatura: Tecnología Web I (SIS-0150) **Docente:** M.Sc. Ing. Jimmy Nataniel Requena Llorentty

Servidor: contenedor `horizonte-servidor-lab02` (Ubuntu 24.04 con systemd, `srv-horizonte`) con los Labs 2 a 4 (Lab 4 en 16/16). Nginx 1.24.0 y OpenSSL 3.0.13. Puertos publicados: 8080→80 y **8443→443**. Archivos en `~/lab05`.

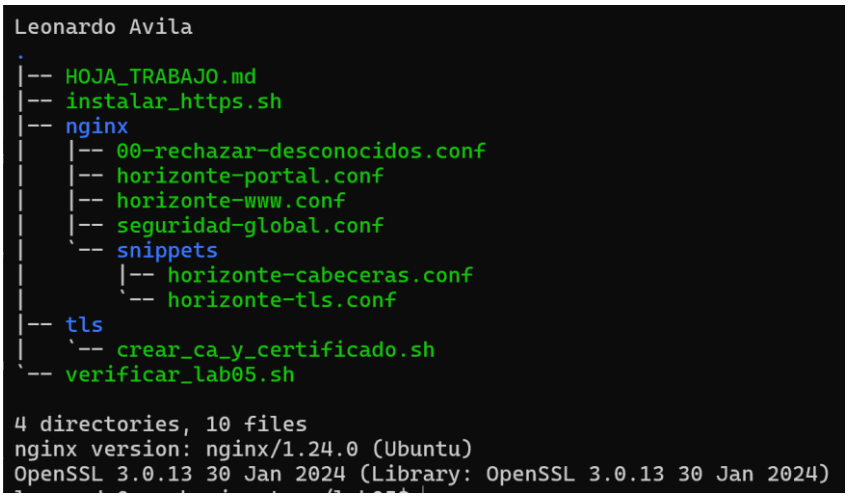


Figura 1. Estructura de ~/lab05, nginx -v y openssl version.

Parte A — Antes: ¿qué ve un espía en HTTP?

En una terminal se dejó escuchando `sudo tcpdump -i lo -A -s0 -l 'tcp port 80' | grep -iE 'usuario|clave'` y en otra se inició sesión en el portal por HTTP con curl (mismo método que `verificar_lab04.sh`: primero se obtiene el token CSRF y luego se envía el formulario a `entrar.php`).

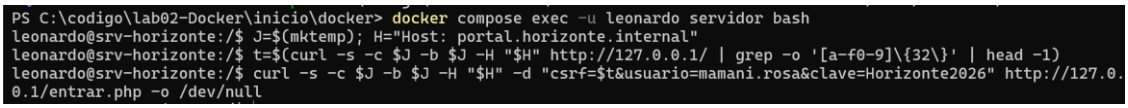


Figura 2. Terminal 2: inicio de sesión por http con curl (token CSRF + POST a entrar.php).

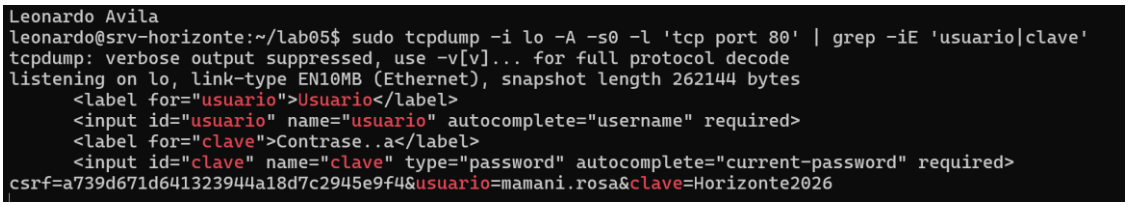


Figura 3. Terminal 1: tcpdump captura csrf=…&usuario=mamani.rosa&clave=Horizonte2026 en texto plano.

¿Qué aparece en tcpdump? El formulario HTML y, sobre todo, el cuerpo del POST completo: `usuario=mamani.rosa&clave=Horizonte2026` y el token CSRF, sin ningún cifrado. **¿Qué podría hacer alguien en el mismo Wi-Fi del colegio?** Capturar el tráfico (sniffing) y leer usuarios, contraseñas y cookies de sesión; con ellos entrar a la cuenta del padre, ver las notas del hijo o secuestrar la sesión, e incluso modificar las respuestas (ataque de intermediario), porque HTTP no ofrece confidencialidad, integridad ni autenticación del servidor.

Parte B — La autoridad certificadora (CA) del colegio

TODO de crear_ca_y_certificado.sh	Solución
1. Nombres en subjectAltName	`subjectAltName=DNS:www.horizonte.internal,DNS:horizonte.internal,D`
2. Vigencia del certificado	`-days 3650` → `-days 90` (los certificados modernos duran 90 días o menos)

Error encontrado: el certificado seguía durando 10 años

En el primer intento solo se cambió el subjectAltName: la línea 40 seguía con `-days 3650` y el certificado salió válido hasta **2036**. Se corrigió con `sed -i 's/-days 3650/-days 90/'` y se volvió a ejecutar el script; la CA no se regeneró (el script solo la crea si no existe `ca.key`), solo el certificado del servidor.

```
Leonardo Avila
17:  openssl req -x509 -new -key ca.key -sha256 -days 1825 -out ca.crt \
33:subjectAltName=DNS:www.horizonte.internal,DNS:horizonte.internal,DNS:portal.horizonte.internal
40:  -days 3650 -sha256 -extfile servidor.ext -out servidor.crt
leonardo@srv-horizonte:~/lab05$
```

Figura 4. Primer intento: SAN correcto, pero la línea 40 aún con -days 3650.

```
Leonardo Avila
leonardo@srv-horizonte:~/lab05$ sudo bash tls/crear_ca_y_certificado.sh
Certificate request self-signature ok
subject=C = BO, O = Unidad Educativa Horizonte, CN = www.horizonte.internal
subject=C = BO, O = Unidad Educativa Horizonte, CN = www.horizonte.internal
issuer=C = BO, ST = Santa Cruz, O = Unidad Educativa Horizonte, CN = Horizonte CA Raiz 2026
notBefore=Oct 12 16:11:31 2026 GMT
notAfter=Oct  9 16:11:31 2036 GMT
X509v3 Subject Alternative Name:
    DNS:www.horizonte.internal, DNS:horizonte.internal, DNS:portal.horizonte.internal
leonardo@srv-horizonte:~/lab05$
```

Figura 5. Certificado del primer intento: notAfter = 2036 (10 años).

```
Leonardo Avila
17:  openssl req -x509 -new -key ca.key -sha256 -days 1825 -out ca.crt \
33:subjectAltName=DNS:www.horizonte.internal,DNS:horizonte.internal,DNS:portal.horizonte.internal
40:  -days 90 -sha256 -extfile servidor.ext -out servidor.crt
leonardo@srv-horizonte:~/lab05$ sudo bash tls/crear_ca_y_certificado.sh
Certificate request self-signature ok
subject=C = BO, O = Unidad Educativa Horizonte, CN = www.horizonte.internal
subject=C = BO, O = Unidad Educativa Horizonte, CN = www.horizonte.internal
issuer=C = BO, ST = Santa Cruz, O = Unidad Educativa Horizonte, CN = Horizonte CA Raiz 2026
notBefore=Oct 12 16:14:06 2026 GMT
notAfter=Jan 10 16:14:06 2027 GMT
X509v3 Subject Alternative Name:
    DNS:www.horizonte.internal, DNS:horizonte.internal, DNS:portal.horizonte.internal
```

Figura 6. Corregido: -days 90 y certificado válido hasta el 10 de enero de 2027 con los 3 nombres.

```
Leonardo Avila
leonardo@srv-horizonte:~/lab05$ sudo openssl x509 -in /etc/horizonte-ca/ca.crt -noout -subject -dates
subject=C = BO, ST = Santa Cruz, O = Unidad Educativa Horizonte, CN = Horizonte CA Raiz 2026
notBefore=Oct 12 16:11:31 2026 GMT
notAfter=Oct 11 16:11:31 2031 GMT
leonardo@srv-horizonte:~/lab05$ sudo openssl x509 -in /etc/horizonte-ca/servidor.crt -noout -issuer -dates -ext subjectA
ltName
issuer=C = BO, ST = Santa Cruz, O = Unidad Educativa Horizonte, CN = Horizonte CA Raiz 2026
notBefore=Oct 12 16:14:06 2026 GMT
notAfter=Jan 10 16:14:06 2027 GMT
[redacted] Subject Alternative Name:
    DNS:www.horizonte.internal, DNS:horizonte.internal, DNS:portal.horizonte.internal
leonardo@srv-horizonte:~/lab05$ sudo ls -l /etc/horizonte-ca/
total 32
-rw-r--r-- 1 root root  830 Oct 12 12:11 ca.crt
-rw------- 1 root root  227 Oct 12 12:11 ca.key
-rw-r--r-- 1 root root   41 Oct 12 12:14 ca.srl
-rw-r--r-- 1 root root 1766 Oct 12 12:14 servidor-cadena.crt
-rw-r--r-- 1 root root  936 Oct 12 12:14 servidor.crt
-rw-r--r-- 1 root root  444 Oct 12 12:14 servidor.csr
-rw-r--r-- 1 root root  305 Oct 12 12:14 servidor.ext
-rw------- 1 root root  227 Oct 12 12:14 servidor.key
```

Figura 7. openssl x509 de la CA y del servidor, y permisos de /etc/horizonte-ca.

Pregunta	Respuesta
1. ¿Cuánto dura la CA?	**5 años** (1825 días): del 12/10/2026 al 11/10/2031. Sujeto: `CN = Horizonte CA Raiz 2026`.
2. ¿Cuánto dura el certificado del servidor y qué nombres cubre?	**90 días**: del 12/10/2026 al 10/01/2027. Emisor: Horizonte CA Raiz 2026. Cubre `www.horizonte.internal`, `horizonte.internal` y `portal.horizonte.internal`.
3. ¿Por qué ca.key nunca sale y ca.crt sí?	`ca.key` es la clave privada que firma certificados: quien la tenga puede emitir certificados falsos para cualquier nombre y todas las PCs que confían en la CA los aceptarían. Por eso queda con permisos 600 (solo root). `ca.crt` solo contiene la clave pública: sirve para verificar firmas, no para crearlas, así que se reparte a las PCs.
4. ¿Por qué no Let's Encrypt para www.horizonte.internal?	Let's Encrypt solo emite para dominios públicos y debe validar que controlamos el dominio desde internet (HTTP-01 o DNS-01). `.internal` es un dominio reservado para redes privadas que no existe en el DNS público, así que no puede validarse: hace falta una CA interna.

Parte C — Nginx con HTTPS y cabeceras

Archivo / TODO	Lo que se escribió
horizonte-tls.conf · 3	`ssl_protocols TLSv1.2 TLSv1.3;` (TLS 1.0 y 1.1 obsoletos, RFC 8996)
horizonte-cabeceras.conf · 4	`add_header Strict-Transport-Security "max-age=31536000" always;`
horizonte-cabeceras.conf · 5	`add_header Content-Security-Policy "default-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline'; script-src 'self'; frame-ancestors 'none'; form-action 'self'; base-uri 'self'" always;`
horizonte-www.conf · 6	`return 301 https://www.horizonte.internal$request_uri;` en el server del puerto 80
horizonte-www.conf · 7	`include snippets/horizonte-cabeceras.conf;` dentro de `location /circulares/`
horizonte-portal.conf · 8	`include snippets/horizonte-tls.conf;` · `include snippets/horizonte-cabeceras.conf;`
00-rechazar-desconocidos.conf · 9	`ssl_reject_handshake on;` en el server 443 default_server
seguridad-global.conf · 10	`server_tokens off;`

La trampa de la herencia: en Nginx, si una `location` declara su propio `add_header` (aquí `X-Robots-Tag` en `/circulares/`), deja de heredar **todos** los `add_header` del `server`, así que esa ruta perdía HSTS y la CSP. Se resolvió volviendo a incluir el snippet dentro de la location. **HTTP/2** se activa con `listen 443 ssl http2;` porque en Nginx 1.24 la directiva `http2 on;` no existe (es de 1.25.1+).



Figura 8. grep de los TODO: cada uno con su directiva debajo.

`sudo bash instalar_https.sh` copió snippets y sitios, instaló la CA en el almacén del propio servidor (`update-ca-certificates`, para que curl confíe sin `-k`), ejecutó `nginx -t` y recargó.

Resultado: 20 de 20 comprobaciones

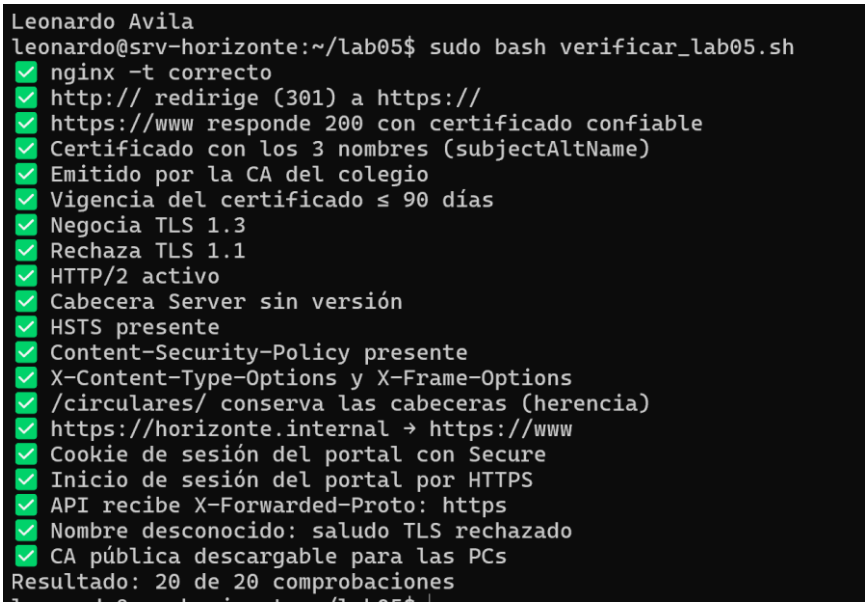


Figura 9. sudo bash verificar_lab05.sh → 20 de 20.

¿Por qué ahora fallan `verificar_lab03.sh` y `verificar_lab04.sh`? Porque prueban por `http://` y esperan contenido (200, páginas de error, login, API), pero ahora el puerto 80 solo responde `301` hacia `https://`. No es un error: es el comportamiento deseado; esos verificadores fueron escritos para un servidor sin HTTPS.

Parte D — Inspecciona la conexión

Comando	¿Qué observas?	¿Quién rechaza?
`echo | openssl s_client -connect 127.0.0.1:443 -servername www.horizonte.internal`	Cadena: 0 = www.horizonte.internal emitido por Horizonte CA Raiz 2026; 1 = la CA. `Verification: OK`, **TLSv1.3**, cifrado `TLS_AES_256_GCM_SHA384`.	— (conexión aceptada)
lo mismo con `-tls1_1`	`no protocols available`: el handshake ni siquiera sale.	**El cliente** (OpenSSL 3 deshabilita TLS 1.1 por su nivel de seguridad)
`-tls1_1 -cipher 'DEFAULT@SECLEVEL=0'`	`tlsv1 alert protocol version`, **alert 70**.	**El servidor** (Nginx solo acepta 1.2 y 1.3)
lo mismo con `-servername intruso.com`	`tlsv1 unrecognized name`, **alert 112**, `no peer certificate available`.	**El servidor** (`ssl_reject_handshake`): no revela su certificado
`curl -sI --resolve www.horizonte.internal:443:127.0.0.1 https://www.horizonte.internal/`	**HTTP/2 200**, `server: nginx` sin versión, HSTS, nosniff, DENY, Referrer-Policy, Permissions-Policy y CSP.	—

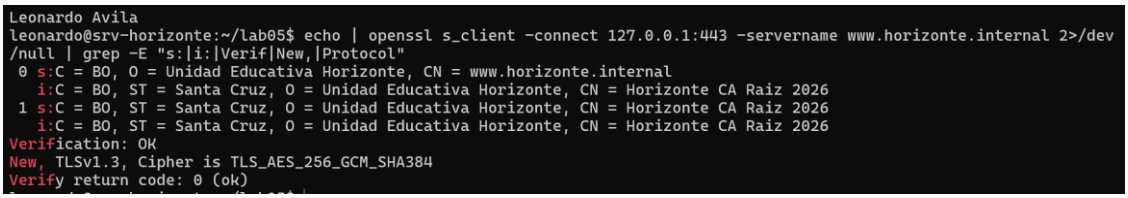


Figura 10. openssl s_client: cadena de 2 certificados, Verification OK y TLS 1.3.

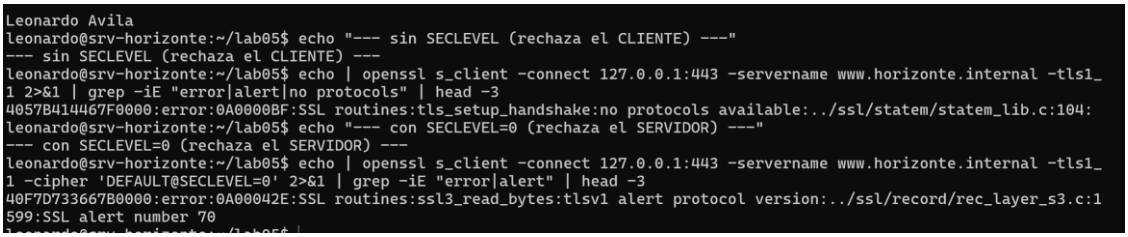


Figura 11. TLS 1.1: "no protocols available" (cliente) frente a "alert number 70" (servidor).

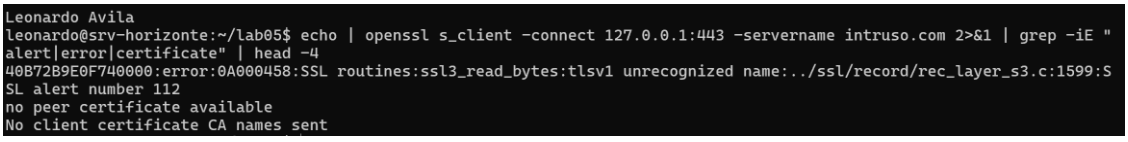


Figura 12. SNI intruso.com: alert 112 y ningún certificado enviado.

```
Leonardo Avila
leonardo@srv-horizonte:~/lab05$ curl -sI --resolve www.horizonte.internal:443:127.0.0.1 https://www.horizonte.internal/
HTTP/2 200
server: nginx
date: Mon, 12 Oct 2026 16:23:25 GMT
content-type: text/html
content-length: 917
last-modified: Fri, 09 Oct 2026 12:50:09 GMT
etag: "6ac8e301-395"
strict-transport-security: max-age=31536000
x-content-type-options: nosniff
x-frame-options: DENY
referrer-policy: strict-origin-when-cross-origin
permissions-policy: camera=(), microphone=(), geolocation=()
content-security-policy: default-src 'self'; img-src 'self' data:; style-src 'self' 'unsafe-inline'; script-src 'self';
frame-ancestors 'none'; form-action 'self'; base-uri 'self'
accept-ranges: bytes
```

Figura 13. curl -sI por HTTPS: HTTP/2 y cabeceras de seguridad.

Parte E — Desde la PC

1. En Docker la CA se copió desde el contenedor: `docker compose cp servidor:/srv/horizonte/www/horizonte-ca.crt C:\codigo\lab05-Docker\horizonte-ca.crt`. Antes de instalarla, Windows la marca como "no es de confianza". **2.** Se instaló con doble clic → Instalar certificado → Usuario actual → Entidades de certificación raíz de confianza.

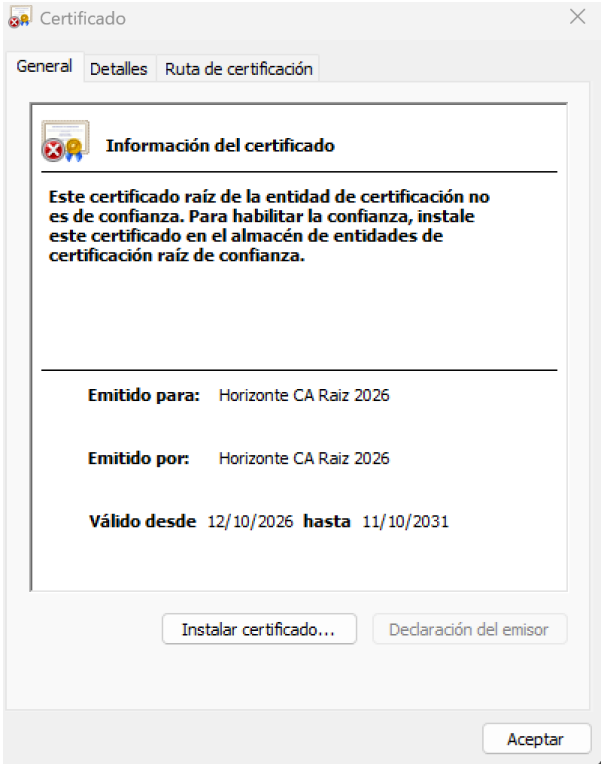


Figura 14. La CA antes de instalarla: Windows aún no confía en ella (válida 12/10/2026 – 11/10/2031).

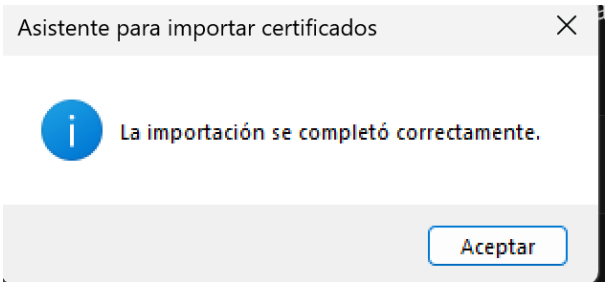


Figura 15. Importación completada en el almacén de raíz de confianza del usuario actual.

3. Se abrió `https://www.horizonte.internal:8443` (en Docker el 443 del contenedor se publica en el 8443). El navegador acepta la conexión sin advertencias y el visor muestra el certificado emitido para `www.horizonte.internal` por **Horizonte CA Raiz 2026**, vigente hasta el 10/01/2027. **4.** En

DevTools > Red, la respuesta trae CSP, HSTS (`max-age=31536000`), `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, Referrer-Policy, Permissions-Policy y `Server: nginx` sin versión.

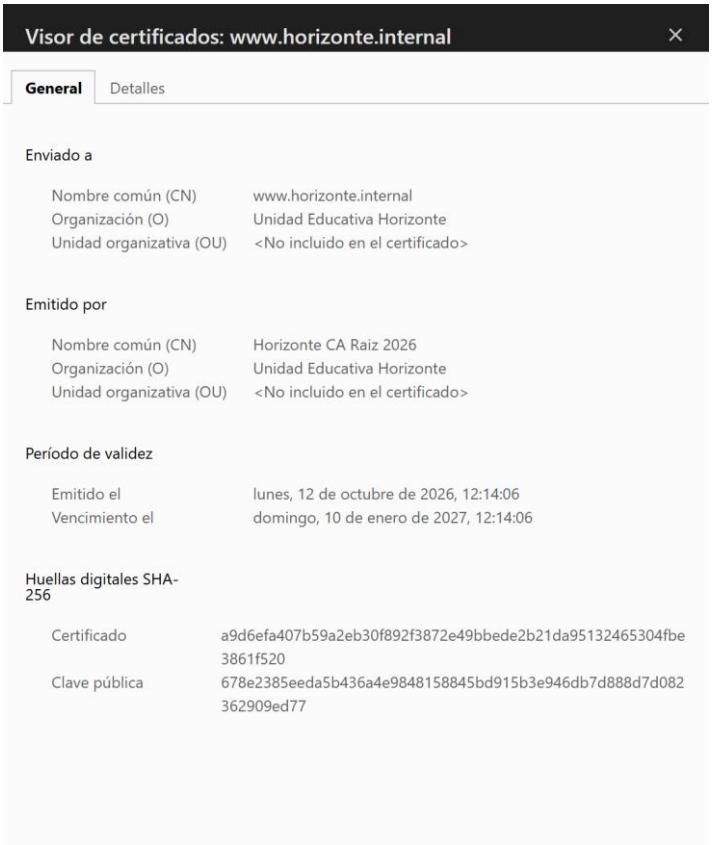


Figura 16. Visor de certificados: enviado a www.horizonte.internal, emitido por Horizonte CA Raiz 2026.

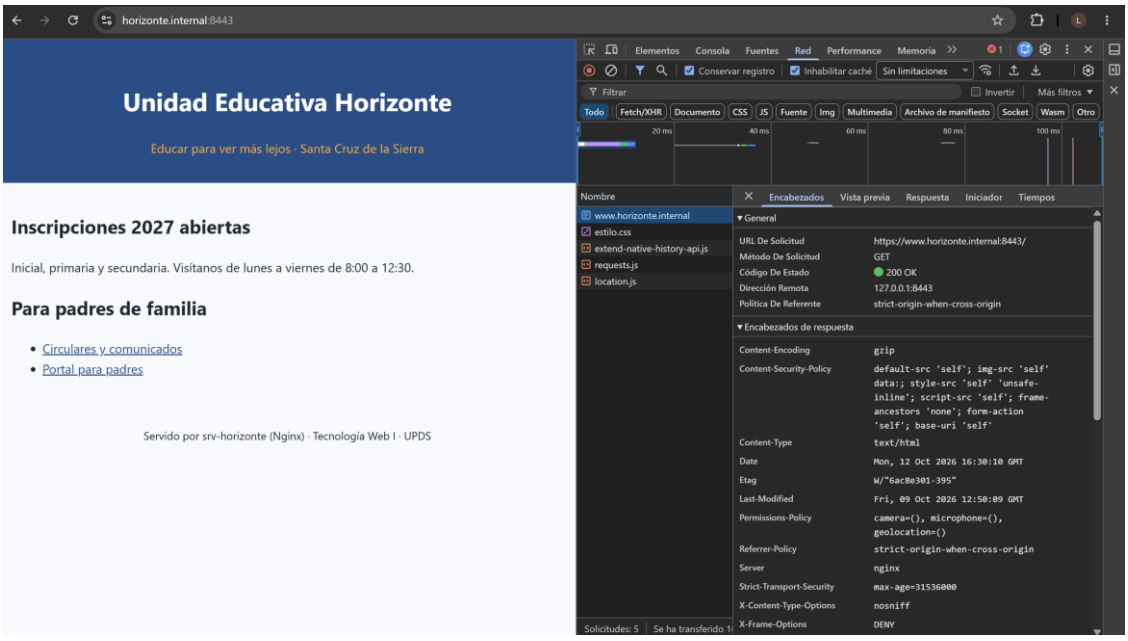


Figura 17. https://www.horizonte.internal:8443 y sus cabeceras de seguridad en DevTools > Red.

Cabecera	srv-horizonte (Lab 5)	upds.edu.bo (Lab 1)
Strict-Transport-Security	`max-age=31536000`	No envía
Content-Security-Policy	Sí (solo recursos propios)	No envía
X-Frame-Options	`DENY`	No envía
X-Content-Type-Options	`nosniff`	No envía
Server	`nginx` (sin versión)	`nginx`

5. ¿Por qué desinstalar la CA al terminar el curso? Porque mientras esté en "raíz de confianza" la PC aceptará **cualquier** certificado firmado con esa CA, para cualquier dominio (incluso un banco). Si alguien obtuviera `ca.key` del servidor de práctica podría suplantar sitios reales ante esta PC sin ninguna advertencia. Además, HSTS hace que el navegador recuerde durante un año que `www.horizonte.internal` solo va por HTTPS.

Reflexión

HTTPS cifra lo que viaja entre el navegador y el servidor, así que nadie en la red puede leer la contraseña como lo hice con tcpdump en la Parte A, y el certificado asegura que estoy hablando con el servidor correcto. Pero no protege todo: si la contraseña es débil se puede adivinar igual, y si el servidor está mal configurado los datos se pueden filtrar aunque la conexión esté cifrada.